META + GOOSE + C++ / V3.0 / 26 OCTOBER 2026

CONTINUUM

The Meta agent repairs its path.

A complete product specification and implementation repository.
Meta reasoning. Bounded actions. Verified outcomes.

OBSERVE	META DECIDES	VERIFY
Path measurements and the task that matters.	Model-selected tools with bounded authority.	Measured outcome and observed restoration.

Specification, prompts, schemas, and typed source with a working first transport.
Implementation follows this handoff.

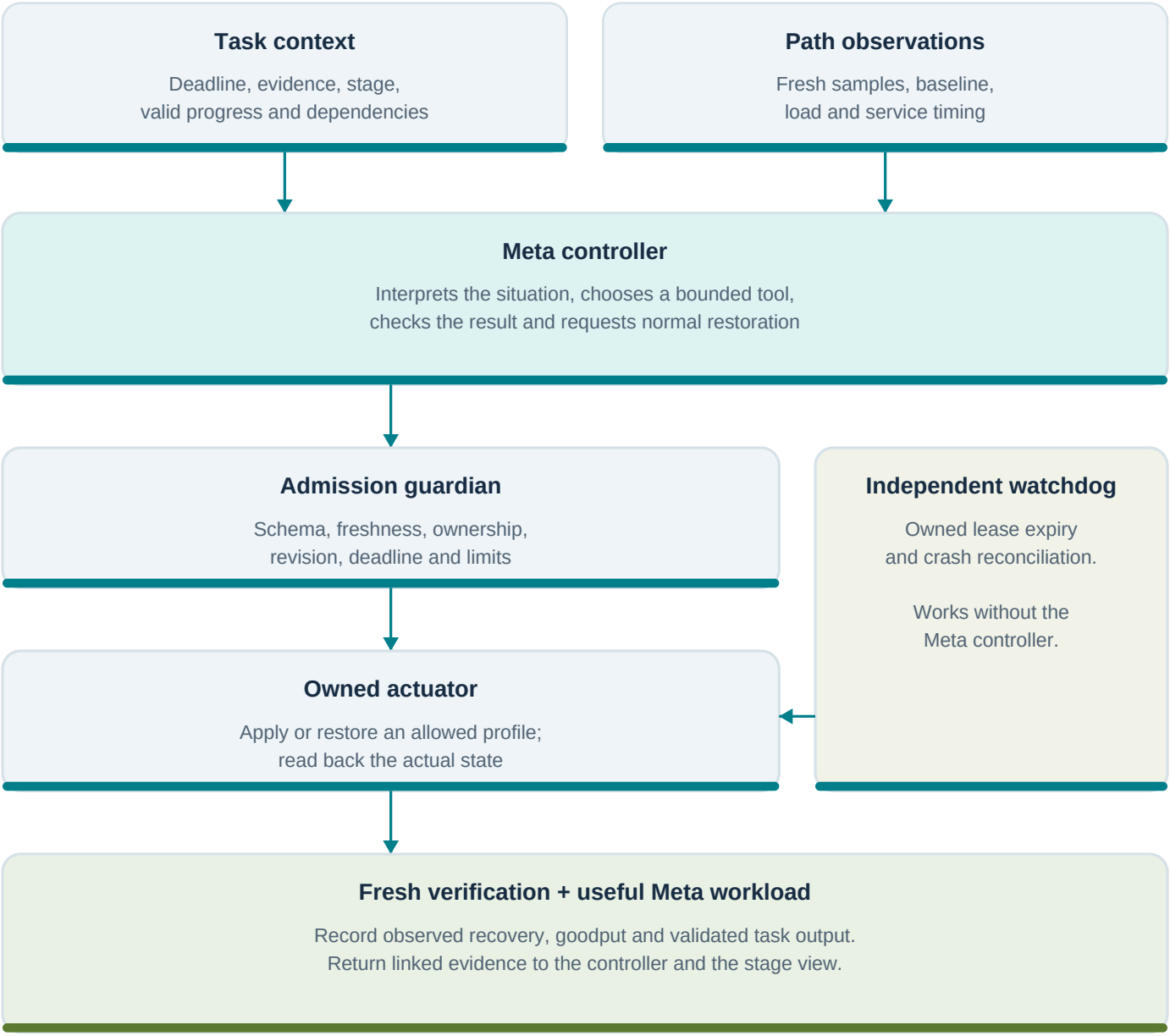
Inside the build

Start with the product and required Meta roles, then follow the architecture, tool contracts, implementation plan, and advanced extension designs. This repository contains the complete specification and the working source described in docs/STATUS.md.

01. The product and its central claim . . . 4
02. Scope and decisions already made . . . 5
03. What the Meta models actually do . . . 5
04. The primary demonstration . . . 6
05. Architecture and authority . . . 8
06. Model protocol and tool contracts . . . 9
07. Data, state, and invariants . . . 10
08. Observation and diagnosis design . . . 11
09. Decision lifecycle and guardrails . . . 12
10. Actuation, ownership, and restoration . . . 13
11. Goose and C++ implementation layout . . . 13
12. Interface and event presentation . . . 14
13. RaptorQ and the advanced transport design . . . 15
14. Durable progress and local Meta execution . . . 16
15. Evaluation and contribution accounting . . . 16
16. Build sequence and acceptance gates . . . 18
17. Delivered files and implementation status . . . 18
18. Source register and unresolved decisions . . . 19

Architecture at a glance

The Meta controller sees both network evidence and task context. It selects a permitted tool; deterministic code admits and executes the request. Independent restoration remains available when the model is unreachable.



Architecture is proposed. All operational source entrypoints in this repository report their implementation state; see docs/STATUS.md. RaptorQ and local inference are separate extension gates.

01. The product and its central claim

Continuum is a Meta-powered agent that understands how network conditions are affecting its work, chooses a bounded intervention, and checks whether that intervention helped the work finish correctly.

The original proposal already has a strong demonstration: create network load, observe latency under load, let a Meta model diagnose the situation and call an apply tool, show the effect, and let the agent undo its change. The upgraded design makes that loop explainable, measurable, and useful to an actual agent workload. It also defines an ambitious route toward reliable evidence delivery and durable task execution.

Meta models are an architectural requirement. The controller's interpretation of the situation and selection of an ordinary network action come from a real Meta model response. A separate Meta workload produces the useful result whose progress the controller protects. Goose is the proposed implementation language; it does not replace model reasoning. C++ is the native integration boundary and the fallback language when a particular Goose integration does not pass its build gate.

The implementation hypothesis is that a model with both network evidence and task context can make better task-level choices than a controller that sees only network signals. That hypothesis must be compared against competent queue management and deterministic control. A successful CAKE demonstration alone establishes the effect of queue management; it does not establish the added value of a model.

The desired outcome is a correct, evidence-backed task result within the run's deadline while retaining useful bulk throughput. A lower ping is supporting evidence. An attractive dashboard is a presentation layer. Neither alone is task completion.

Product promise to demonstrate

“The Meta agent notices that its environment is undermining its work, chooses an allowed repair, proves what happened, and restores the environment afterward.”

This phrasing is a target for the demo. No speedup, autonomous recovery rate, or completed live model integration is claimed by this repository.

02. Scope and decisions already made

Decision	Specification
Deliverable now	Complete design documents plus working sim, contract suites, virtual-router loop and Meta HTTPS transport; controller loop and live integration follow in ticket order.
Intelligence	Meta models perform the controller reasoning and useful workload reasoning.
Primary language	aardappel's Goose programming language, pinned for reproducible builds.
Native integration	C-compatible ABI implemented in C++ where native libraries or OS APIs require it.
Fallback language	C++ behind the same contracts, or a C++ application if the primary-language gate fails.
Required demo	Observe, diagnose, apply a bounded profile, verify, and restore; useful Meta workload continues.
Advanced direction	RaptorQ, task-aware transport, exact cache/resume, eligible local Meta execution, and durable recovery.
User interface	Small event-and-metrics view; a specific web framework is not selected by this repository.
Development reference	Synthetic fixtures are labeled; they are not live Meta responses or performance evidence.

The original concept's ten-day horizon is retained as a planning assumption. The actual event deadline, team size, hardware, account access, available credits, and judging rules are unresolved inputs. They belong in the first build gate, not in hidden assumptions scattered through code.

The owner's reframing notes encourage ambitious design while preserving the operator's goal and success criteria. That means designing a stronger system and a credible route to it while delivering the specified repository. It does not expand this deliverable into a completed implementation.

03. What the Meta models actually do

Required controller role

The controller receives recent path observations, an explicit task summary, available profiles, current ownership and lease state, and recent action outcomes. It chooses whether to gather more evidence, hold the current configuration, request a permitted profile, verify an action, or restore its owned configuration. Each consequential request must cite observation identifiers and provide a short operational rationale.

The controller is responsible for distinguishing plausible situations: queueing under access-link load, provider processing delay, destination-specific trouble, host pressure, and insufficient or stale evidence. It can be wrong; schema checks, authority checks, and independent outcome verification constrain the consequences. The evaluator records both useful and mistaken decisions.

Required workload role

A separate model context performs meaningful application work. The proposed default is a release-readiness audit using versioned evidence documents. The workload must interpret conflicting evidence, identify blockers, cite source identifiers, and produce a structured decision. It cannot be replaced by a prerecorded success string when presenting a live Meta demonstration.

The two roles can use the same approved Meta model ID with different prompts and conversation state. This avoids making a second paid model a prerequisite. The controller sees task constraints and progress, not the workload's unrestricted private conversation.

Optional roles

Muse Spark can review completed incident traces and propose policy experiments. A local Meta open-weight model can execute an eligible semantic step after hardware, quality, and availability checks. Neither is required for the first closed loop. A hosted open-weight endpoint is still a remote dependency and does not establish offline operation.

Model selection and access

The original proposal names Muse Wonder as the primary controller. The official Meta documentation checked for this project documents `muse-wonder-1.0`, the Meta Model API base URL, and a Responses protocol. Configuration keeps the actual model ID explicit. Event eligibility and the user's account access still require a real preflight. Documentation availability is not evidence that an account has permission or credits. See the source register and MODEL_INTEGRATION.md.

There is no automatic fallback to a non-Meta provider. If the selected Meta model is unavailable, the future application reports that condition and follows its bounded wait/restore behavior. A change of approved Meta model is a recorded configuration decision.

04. The primary demonstration

The stage should show one understandable causal loop. Begin with a live Meta workload on a controlled path. Start a bulk transfer that makes the path's latency under load rise. The Meta controller reads independent observations and the workload's deadline/progress, calls an allowed profile tool, receives the actual result, and then checks new measurements. Show the workload's valid result and the agent's restoration request.

The demonstration must retain the same workload, validator, model configuration, topology, and relevant resource budgets across comparisons. A live run can illustrate behavior; repeated matched runs establish the numerical claim. Preserve both in the evidence export.

Minimum visible sequence

1. Show the selected Meta model ID, live/synthetic mode, workload, deadline, owned interface, and initial profile.
2. Show a recent idle observation and normal workload progress.
3. Introduce the recorded load scenario without changing the workload.
4. Show the controller's tool reads, concise diagnosis, and chosen profile request.

5. Show the guardian's admission decision and the actuator's observed result separately.
6. Show the post-action measurement window, task progress, and bulk goodput together.
7. Show the workload's structured result and independent validation outcome.
8. Show the Meta controller call `restore_profile`, followed by observed restoration. Show the watchdog as a separate failure drill.

“The model requested an action,” “the tool applied it,” and “the action improved the outcome” are three different events. The interface must preserve those distinctions.

Proposed meaningful workload

The included synthetic release corpus describes a release candidate with two blockers: REL-17 has no completed rollback rehearsal, and SEC-42 is an unresolved high-severity authorization defect. A plausible-sounding deploy recommendation must fail validation even if it arrives quickly. The expected decision is `hold` with both blockers and source evidence.

The proposed eight logical stages are: read the manifest; retrieve CI evidence; retrieve security evidence; retrieve rollout policy; retrieve support readiness; ask Meta to reconcile the evidence; validate the structured report; commit the result. The retrievals are ordinary workload operations, not the controller's six tools. A production task adapter can later parallelize independent reads; the comparison must use the same execution strategy in all arms.

The fixture is a proposed default for demonstrating real reasoning. It is easy to replace with another useful task that has a reproducible corpus, an objective validator, and enough network sensitivity to expose the intended problem.

05. Architecture and authority

Component	Responsibility	Boundary
Meta controller	Diagnose from observations and task context; choose allowed actions; inspect outcomes.	Produces typed tool calls, not shell commands.
Meta workload	Interpret versioned application evidence and generate a structured useful result.	Uses workload-specific tools and validator.
Goose coordinator	Manage model turns, deadlines, run identity, tool dispatch, and event order.	No privileged network access in the main process.
Collector	Sample path, service, host, queue, and workload signals with provenance.	Unknown and missing data remain explicit.
Admission guardian	Enforce schema, scope, ownership, freshness, limits, and revisions.	Validates an action; does not impersonate the model's choice.
Actuator helper	Apply an allowlisted profile on an owned lab interface and read back state.	Narrow local privilege and finite operations.
Independent watchdog	Restore an expired owned lease and reconcile crash state.	Operates without a model response or coordinator survival.
Event/receipt store	Record model requests, tool results, measurements, and validation.	Durable implementation is a later build ticket.
Stage view	Present measurements, decisions, and results with provenance.	Read-only by default; operator controls are distinct.

The required controller path is observation plus task context, then Meta decision, then admission, then actuation, then fresh verification, then a returned tool result or follow-up model turn. The safety path is lease expiry or a recognized restoration condition, followed by local reconciliation and restoration. The safety path must remain available when the Meta API is slow or unreachable.

This separation does not make the Meta model decorative. Normal apply/restore choices in the required live demonstration must have a recorded model-origin tool call. Emergency expiry restoration is attributed to the watchdog, and reported as such.

06. Model protocol and tool contracts

The provider adapter is specified as native HTTP/TLS behind a stable boundary. The coordinator constructs a Meta Responses request, submits it, parses completed output items, validates function arguments, executes registered tools, and returns each tool result using the original call identifier. The API key is read through an environment reference and is excluded from prompts, fixtures, logs, and version control.

The supplied tool definitions use the documented flat Responses function shape. The application must enforce its own tool-turn count, request deadline, response size, token budget, and tool-execution limits. Do not assume that provider response syntax supplies application authorization or idempotency. Exact provider semantics and current documentation links are in MODEL_INTEGRATION.md.

Core tool	Required purpose	Result meaning
read_observation	Fetch fresh observations for the declared path and run.	Values, time window, provenance, and explicit missing fields.
read_task_context	Read current task constraints and progress.	Deadline, current stage, dependencies, and execution limits.
list_profiles	Discover the finite profiles this run may request.	Profile IDs, bounds, expected revision, scope, and eligibility.
apply_profile	Request one allowed profile against current evidence and revision.	Applied, rejected, unknown, or failed; never inferred from prose.
read_action_result	Inspect an action receipt and post-action measurements.	Application state and verification state remain separate.
restore_profile	Request restoration of an owned active lease.	Restored, no-op, conflict, unknown, or failure with observed state.

A normal natural-language model response that requests no tool is a valid outcome. It must not be silently converted into a tool call. An unknown tool, malformed argument object, stale revision, contradictory scope, or out-of-budget new profile request produces a structured error and no mutation. Owned restoration is exempt from apply budgets and freshness gates, as specified in section 09.

Read-only tool calls may later run concurrently if their snapshots are consistent. Mutating calls are serialized per owned interface. If a model emits multiple mutations in one response, the dispatcher must not apply all of them concurrently. The build chooses an explicit rejection or ordered admission policy and verifies each state transition.

Only concise operational explanations are required in the event log: relevant evidence IDs, a working diagnosis, why the selected profile is appropriate, and what result would trigger restoration. The application does not request or depend on hidden chain-of-thought.

07. Data, state, and invariants

Versioned JSON schemas are the wire and fixture specification. Goose records are the internal domain types. `src/tooljson.goose` and `src/meta_shape.goose` map between them field by field, rejecting malformed input before constructing an actionable internal record. A schema-valid object can still be semantically invalid; authorization, freshness, cross-field constraints, and state transitions need runtime checks.

Record	Required information
Observation	Run/path identity, route epoch, observation ID, capture window, baselines, measured values, missing-data indicators, and provenance.
Task context	Task/run identity, revision, current stage, deadline, evidence requirements, remote dependencies, and recent valid progress.
Action request	Unique request ID, evidence references, profile or lease ID, expected revision, lease bound, and operational rationale. Admission expiry is derived from the bound controller request.
Tool result	Originating call/request ID, explicit status, observed revision, action receipt if any, and structured error or measurements.
Workload result	Decision, supported findings, evidence references, and a validation-compatible structured output.
Event	Run/session identity, ordered sequence, monotonic elapsed time, event type, origin, and linked record IDs.

Run-local monotonic time governs elapsed-time comparisons. Wall time is for human-readable reporting. Monotonic timestamps from a previous process session cannot be compared directly with a new session after restart. A future durable implementation records session/boot identity and reconciles expiry against a restart-safe policy.

Non-negotiable invariants

- A model tool call is not proof of an applied action.
- A successful profile application is not proof of recovered task performance.
- Missing observations are not measurements of packet loss or an outage.
- Input freshness and route identity are checked again before a new profile application. Restoration checks owned-lease identity, current revision, and restorable state without requiring fresh network evidence.
- The application owns only the declared lab profile/interface; it cannot overwrite unrelated changes during restoration.
- Duplicate action requests cannot produce duplicate effects; a retry receives the previous receipt or an explicit unresolved state.
- Quality validation precedes task completion, including after a model switch or recovery.
- Synthetic observations and generated example responses remain visibly labeled.

08. Observation and diagnosis design

The collector should combine multiple signals rather than feed a single ping threshold to the model. Useful observations include a local gateway reference, multiple remote reference probes, an application echo over the same controlled leg, bulk throughput, owned-queue statistics when available, and model/tool request timing. Record sample counts, the capture interval, path identity, and clock/session information.

Separate network time from provider processing as far as instrumentation allows. Client request timing alone does not prove which segment was slow. A controlled echo can help identify path delay; server-side timing or provider metadata can supply additional context when available. Report residual uncertainty instead of labeling every long model response as bufferbloat.

Evidence pattern	Model hypothesis to consider	Bounded next step
Stable gateway; remote probes and application echo rise with sustained access load	Queueing on the controlled path is plausible.	Inspect current profile and request a limited trial if authorized.
Echo stable; provider response time rises	Service or inference delay is plausible.	Preserve the network profile; wait or revise workload scheduling.
Gateway also slow; host/WLAN symptoms	Local host or LAN problem is plausible.	Gather evidence; avoid claiming a WAN shaping remedy.
One destination changes; other probes remain stable	Destination-specific issue is plausible.	Limit the diagnosis to that dependency.
Probes are missing, stale, or cross a route change	The evidence is insufficient.	Refresh the snapshot and invalidate outstanding proposals.

The future policy must define sampling windows, minimum successful observations, stale-data limits, verification windows, and cooldowns in configuration. Initial values are calibration choices, not universal constants. The evaluator records them and freezes them before measured comparisons.

The collector must not read the impairment generator's secret schedule as an input to the model or baseline controller. Otherwise, the demonstration leaks ground truth and overstates diagnostic capability.

09. Decision lifecycle and guardrails

The intended controller states are idle, observing, awaiting_model, admitting_action, applying, verifying, holding, restoring, and degraded. Failure states retain enough information to decide whether an action is known to have happened. The state machine must distinguish an HTTP timeout from a rejected tool call and from an actuator timeout with an unknown outcome.

Admission sequence

These freshness, cooldown, and apply-budget checks govern a new profile change. Restoration of an owned active lease is exempt from the apply budget, cooldown, and observation-age gates. It still requires correct ownership, current revision, and a restorable state.

1. Parse the completed tool arguments and enforce the schema and size limits.
2. Resolve the tool name, authenticate the caller, and validate run/scope identity and allowed operations.
3. Look up the request ID within that scope. Return the historical receipt for an exact replay; reject identity reuse with different parameters. Reconcile an unresolved prior effect instead of reexecuting it.
4. For a new apply, match path, route epoch, interface ownership, and expected configuration revision.
5. Verify observation age, minimum evidence, admission expiry, and the remaining deadline for that new apply.
6. Check the profile allowlist, rate floor, lease duration, cooldown, and apply budget.
7. Record the new action intent and dispatch to the narrow helper.
8. Observe and store the helper result before returning a model-facing receipt.

The guardian can reject an otherwise plausible model request. It should return a structured reason that the model can use to gather better evidence or choose a different permitted action. It must not fabricate a successful result to keep the conversation flowing.

A minimum verification interval and a useful-goodput floor prevent declaring a victory by simply throttling all traffic. The model can request restoration after an ineffective or harmful trial. Lease expiry and a narrowly defined local safety policy can also initiate restoration without waiting for the model.

The degraded state is an explicit operating mode. If the cloud controller is unreachable, the system can retain a currently valid lease until its permitted expiry, restore locally when required, and record that no new Meta decision was available. More advanced offline reasoning is an extension, not a hidden assumption in the core demo.

10. Actuation, ownership, and restoration

The demo actuator targets an isolated Linux lab interface or another explicitly owned controllable path. It exposes finite profiles such as baseline and a calibrated interactive SQM profile. The model selects a profile ID. It does not supply arbitrary `tc` arguments, interface names, executable paths, shell fragments, or unbounded rates.

Each mutation uses an ownership identifier, expected revision, action ID, and lease. The helper verifies the current owned state before applying the requested profile, reads back the result, advances the revision, and emits a receipt. Restoration uses the saved owned baseline and checks for conflicts rather than overwriting an unrelated operator change.

The independent watchdog has a separately scheduled lifetime and access to the minimum durable lease information needed to restore. A timer inside the coordinator is insufficient for crash recovery. The future implementation must exercise process death, expired leases, repeated restoration, unknown helper outcomes, and conflicting external changes.

The lab document specifies topology and ownership but deliberately does not run privileged network commands. The router HTTP helper is implemented and proven against the lab double; SSH/tc backends and real-router discovery are later build tickets. Remaining native stubs return an explicit unimplemented status.

An application on an ordinary laptop cannot assume authority over an upstream ISP queue. The controlled lab makes the reachable actuator and the induced impairment explicit. Future deployment on a router or gateway is a separate product decision.

11. Goose and C++ implementation layout

The selected Goose is the language at `github.com/aardappel/goose`. The repository pins the compiler release in TOOLCHAIN.lock.json. It includes module boundaries for contracts, configuration, provider requests, controller coordination, telemetry, guardian admission, actuator calls, task execution, event recording, and optional transport.

Native I/O should cross a C ABI with documented ownership and lifetime rules. Use plain fixed-width fields, opaque handles, explicit lengths, and copy-or-borrow contracts. C++ exceptions must not cross the ABI. Network callbacks should enqueue bounded native events; Goose should poll or drain copied events instead of receiving arbitrary callbacks into state whose lifetime is unclear.

Layer	Proposed implementation	First gate
Domain records and coordinator	Goose	Type checking and an explicit state/status interface.
Meta HTTP/TLS adapter	C++ native library behind C ABI	One authenticated request, bounded response parsing, cancellation, and correct tool-result replay.

Layer	Proposed implementation	First gate
JSON/schema mapping	Selected native parser or Goose implementation	Reject malformed and cross-field-invalid input; map every required field.
Collector and actuator IPC	Native OS adapter plus Goose contracts	Controlled readback, identity, ownership, and failure semantics.
Durable events/leases	Native database/filesystem adapter	Crash-safe write ordering and restart reconciliation.
Optional QUIC/codec	Native adapter	Independent build, lifecycle, interoperability, and budget gates.

The C++ fallback skeleton preserves the public contract and reports unimplemented operations. It is not a second complete application. If an integration is blocked, decide at the documented gate whether to implement that adapter in C++ or move the application to C++. Do not introduce Python or Rust as an unannounced product runtime.

The provided publishing utility uses Python to create the PDF. That utility is documentation tooling and is outside the proposed application runtime.

12. Interface and event presentation

Use a compact view with five visible areas: current task/result, path measurements, Meta controller decisions, action lifecycle, and the comparison outcome. The view must show live/synthetic mode, model ID, active configuration revision, evidence age, lease expiry, and whether a displayed number is a current sample or a completed-run statistic.

The decision stream should read naturally: “Observed rising application echo under load; requested interactive profile; guard admitted revision 4; helper observed revision 5; verification still pending.” A later event can say whether the trial met its acceptance criteria. Show concise evidence-backed rationales rather than a scrolling wall of model text.

Progress increments only when the workload's declared completion boundary is reached. Request dispatch, received tokens, decoded transport bytes, and successful tool execution can be useful activity indicators, but they are not all accepted task progress. Keep those counters separate.

Operator controls are explicit: start a configured run, start a known load scenario, pause at a defined boundary, request restoration, and export the record. A manual action is attributed to the operator. It cannot masquerade as a Meta decision in the stage recording.

The exact interface technology remains open. The first implementation can be a terminal view if it exposes the required evidence clearly. A polished web UI is optional after the working loop and measurements exist.

13. RaptorQ and the advanced transport design

RaptorQ is specified as an optional capability for delivering exact evidence objects across a lossy leg that the project controls at both ends. It can generate repair symbols so a receiver can recover missing source data once sufficient independent symbols are available. Its value must be measured against a competent reliable transport under matched resources. It is not a general promise of faster internet.

The proposed placement is client to an owned relay or tool service. The relay reconstructs and validates the object before it is used in a Meta request or returned as tool evidence. The Meta API's ordinary TLS connection remains an ordinary provider connection. Continuum cannot insert a private RaptorQ wire protocol into a third-party endpoint that does not implement it.

Exact-object contract

An object descriptor records content identity, total length, content hash, source version, expiry, codec parameters, allowed symbol range, and maximum resources. Reliable authenticated control carries the descriptor and completion/cancellation messages. A proposed QUIC DATAGRAM leg carries bounded symbols, subject to negotiated payload limits and congestion control. The receiver deduplicates symbols, enforces object limits, reconstructs, and verifies the exact bytes before making the object available.

Distinguish object decoded, hash verified, evidence accepted, and task result committed. Coding success does not imply that the evidence is fresh or that the model's answer is valid. A hash identifies bytes; it does not authorize their origin. Authentication and source binding remain separate requirements.

Shared budget and deadlines

Count source symbols, repair symbols, repeated sends, control traffic, and transport overhead in a clearly stated cumulative budget. A cap on distinct repair identifiers alone does not cap wire bytes. The scheduler stops admitting symbols when there is insufficient time for delivery, decoding, input verification, model work, and final validation before the task deadline.

Coding must coexist with the transport's congestion-control obligations. When queue pressure is the dominant problem, additional repair bytes can make it worse. Begin with bounded deterministic admission and evaluate the loss regimes where it helps. A later model can choose a permitted object-delivery policy, while the runtime enforces resource limits.

Candidate native stack and build decision

MsQuic and an RFC 6330-compatible native RaptorQ implementation are research candidates behind the C ABI. Pin their exact versions, review licenses and buffer lifetimes, and test metadata interoperability before selecting them. This repository does not vendor a codec or claim a QUIC connection. Detailed selection gates and alternatives are in ADVANCED_ROADMAP.md.

The decisive advanced scene is a real evidence object crossing the controlled leg under loss, reconstructing exactly, then enabling an actual Meta workload step. A local encode/decode demonstration is a useful codec test but is not

this scene.

14. Durable progress and local Meta execution

These extensions make the project more than a one-off network repair demonstration, but they are gated behind the core loop. A task step declares its inputs, source versions, freshness rules, validator, deadline, remote dependencies, and side-effect behavior. The runtime stages a result, validates it, and commits it before it can advance durable progress.

On restart, reuse only committed results whose dependencies remain valid. An external action with an unknown outcome must be reconciled using an idempotency key or a readback operation; blindly replaying it can duplicate effects. The release-audit demonstration can begin with read-only evidence retrieval to avoid inventing a full transactional workflow platform for the hackathon.

Local execution requires more than a model name. The required inputs must already be available and fresh, the local model must support the task, the device must meet resource and latency constraints, and the output must pass the same acceptance standard. If any mandatory remote dependency remains, calling a local model does not make the step offline-capable.

A future route decision can choose remote Meta, an eligible local Meta model, cached exact work, or checkpoint-and-wait. The model may propose the choice; the runtime verifies eligibility. The interface must report when a checkpoint was actually persisted instead of presenting a policy label as a completed durable write.

Content-addressed cache/resume, dependency-aware admission, bounded speculative pure reads, and real multipath delivery are successive research layers. Each adds a falsifiable benefit and a resource cost. None is necessary to claim the initial Meta-controlled repair loop works.

15. Evaluation and contribution accounting

Separate three questions. First, does queue management improve the controlled impairment? Second, does the implemented Meta controller choose and verify appropriate actions? Third, does task-aware reasoning add value over competent deterministic control with the same available actions? The experiment must not answer the first question and present it as proof of all three.

Arm	Controller and network policy	What it establishes
A	Unmanaged known baseline; same Meta workload.	Sensitivity of the workload to the induced impairment.
B	Calibrated static SQM; same Meta workload.	The benefit already available from ordinary queue management.
C	Competent deterministic adaptive controller; same profiles, sensors, budgets, and workload.	A meaningful baseline for the Meta controller.

Arm	Controller and network policy	What it establishes
D	Meta controller with task context; same profiles, sensors, budgets, and workload.	The proposed model-assisted decision contribution.
E	Same Meta controller with task context withheld.	Whether task context changes useful choices.

The task-blind ablation keeps the model, prompt budget, tool set, and permissible network actions as comparable as possible. Record unavoidable differences. Do not weaken the deterministic baseline by withholding signals it would reasonably use. Calibrate baseline parameters separately and freeze them before evaluation.

The advanced phase adds a strong independent composition baseline: use the best calibrated network controller alongside the same transport, cache, model, durable runtime, and resource budgets, but without cross-layer coordination. Compare against coordinated Continuum to isolate the value of coordination. Reliable transport versus RaptorQ is a separate matched experiment.

Required measurements

- Valid workload completion by deadline, with all started runs in the denominator.
- Completion-time distribution, plus explicit failures and timeouts.
- Network RTT and application echo distributions with counts and capture windows.
- Provider/model elapsed time and controller overhead, reported separately where observable.
- Bulk useful goodput and total transmitted bytes.
- False or ineffective interventions, profile churn, and restoration outcomes.
- Model/tool requests, token usage where available, and abandoned work.
- Quality failures, unsupported findings, and missing expected blockers.

Use randomized or paired scenario ordering, identical evidence corpora, warm/cold state controls, and raw event exports. Choose repetition counts and analysis methods before inspecting the result. A small hackathon sample can support a carefully bounded demo claim; it does not establish general performance across real networks.

A falsifying result is valuable: if static SQM solves the problem equally well, or controller inference delays exceed the benefit, narrow the claim and revise the design. Do not add artificial round trips or hide failed runs to make the model appear necessary.

16. Build sequence and acceptance gates

BUILD_PLAN.md provides ticket IDs, dependencies, outputs, and exit checks. The sequence puts actual Meta integration at the beginning. The core model must not be postponed until after an extensive infrastructure build.

Gate	Required artifact before moving on
G0: Access and contracts	Approved Meta model/account, chosen lab host, pinned compiler, tool definitions, and explicit run limits.
G1: Model tool loop	One real Meta response selects a read-only tool and receives its correctly linked result; malformed calls are rejected.
G2: Owned actuator	Apply, readback, restore, expiry, and conflict behavior on an isolated interface.
G3: First closed loop	A real Meta-selected action follows genuine observations, changes owned state, and receives verification evidence.
G4: Useful task	Real Meta workload output passes the corpus validator under the controlled scenario.
G5: Comparative evidence	Matched baseline/controller runs and negative controls, with raw records and honest uncertainty.
G6: Stage readiness	Repeatable demonstration, cleanup, model attribution, and a labeled backup recording.
G7: Advanced extension	Only after G3/G4 and the required negative controls, a bounded optional local-model or coded-object experiment with its own acceptance test.

A scaffold syntax check is not G1. A fake tool response is not G2. A fixed fixture that always requests a profile is not G3. A decoder recovering local bytes is not G7's live transport demonstration. The gate language is deliberately concrete so the eventual implementer can tell what is missing.

If the available time is shorter than the assumed ten days, reduce optional features and presentation polish. Preserve the real Meta decision, actual bounded action, independent verification, restoration, and useful workload. Those define the project's identity.

17. Delivered files and implementation status

This repository is the complete handoff. Start with README.md and this specification, then MODEL_INTEGRATION.md, IMPLEMENTATION_MAP.md, and BUILD_PLAN.md. The schemas and prompts provide exact starting interfaces. The synthetic corpus and examples make the desired behavior reviewable before live integration is complete.

The Goose modules and C++ fallback expose explicit results for what is not yet implemented. The status entry point reports the implementation ledger; sim, contract suites and the virtual-router loop run green. The Meta HTTPS adapter is implemented but has made no live call. The native boundary is a declared C ABI with a working router backend, a working Meta backend, and stubs where backends are still pending - not a finished actuator for every

path.

STATUS.md records exactly which checks were performed on this repository. Planned acceptance tests are distinguished from executed structural checks. Earlier experimental implementations are not imported as the primary application and do not determine the architecture of this repository.

Handoff checklist

- Product thesis, constraints, original-source alignment, and Meta contribution are specified.
- Model roles, operational prompts, provider semantics, and tool schemas are present.
- Architecture, state transitions, admission checks, and restoration responsibilities are defined.
- Goose module boundaries and native C ABI/fallback stubs are provided.
- Configuration examples and synthetic evidence/response fixtures are labeled.
- Lab topology, evaluation controls, build tickets, demo flow, and pitch are included.
- RaptorQ and later continuity capabilities have explicit placement, budgets, and gates.
- Live model calls and measured hackathon performance remain future implementation work, tracked ticket by ticket in STATUS.md.

18. Source register and unresolved decisions

The preceding inputs are the original hackathon concept (Meta-driven closed-loop demo) and the owner's reframing notes on ambition and problem-framing. This specification incorporates the Goose-first/C++ fallback preference and the implement-the-loop working method.

Primary technical sources checked or carried into the research register include:

- Meta Model API overview: https://dev.meta.ai/docs/overview
- Meta model catalog: https://dev.meta.ai/docs/models
- Meta tool calling: https://dev.meta.ai/docs/tool-calling
- Meta Responses protocol: https://dev.meta.ai/docs/protocols/responses
- Goose language and official releases: https://github.com/aardappel/goose
- RaptorQ, RFC 6330: https://www.rfc-editor.org/rfc/rfc6330
- QUIC DATAGRAM, RFC 9221: https://www.rfc-editor.org/rfc/rfc9221
- FEC and congestion-control considerations, RFC 9265: https://www.rfc-editor.org/rfc/rfc9265
- QUIC loss detection and congestion control, RFC 9002: https://www.rfc-editor.org/rfc/rfc9002
- CAKE research: https://arxiv.org/abs/1804.07617
- cake-autorate implementation: https://github.com/lynxthecat/cake-autorate
- MsQuic repository and documentation: https://github.com/microsoft/msquic

These sources support the relevant protocol and implementation choices; they do not validate Continuum's proposed performance. MODEL_INTEGRATION.md and SOURCES.md separate documented provider behavior from account-specific preflight and project design decisions. ADVANCED_ROADMAP.md explains the research status of optional layers.

The remaining decisions are the actual event date and rules, approved Meta model/account, available Linux host and ownership scope, measured link capacities, team capacity, workload selection, UI technology, and whether any advanced extension fits after the core gates. Each has a concrete first-build decision point. None is silently converted into an implemented capability by this repository.